كتاب "بَرمَجة": دليلك الشامل لتعلم البرمجة باللغة العربية **حقوق النشر محفوظة للمبدع: علاء صابر** ## مقدمة أهلاً بك في عالم البرمجة المثير! هذا الكتاب هو بوابتك لتعلم أساسيات البرمجة باستخدام لغة "بَرمَجة"، وهي لغة برمجة عربية بالكامل، مصممة خصيصًا لتجعل رحلتك في عالم البرمجة سهلة وممتعة. إذا كنت تتساءل كيف تعمل التطبيقات والمواقع التي تستخدمها يوميًا، فهذا الكتاب سيمنحك الأدوات اللازمة لبناء تطبيقاتك الخاصة. "بَرمَجة" ليست مجرد لغة جديدة، بل هي جسر يربط بين جمال اللغة العربية وقوة البرمجة، مما يزيل حاجز اللغة الذي قد يواجهه الكثيرون عند تعلم لغات البرمجة الأجنبية. ستتعلم هنا المفاهيم الأساسية للبرمجة بكلمات عربية مألوفة، ثم ترى كيف تتحول هذه الكلمات إلى كود JavaScript قياسي يمكن تشغيله في أي مكان. دعنا نبدأ هذه الرحلة معًا! ## الفصل الأول: ما هي البرمجة ولماذا "بَرمَجة"؟ ### 1.1 ما هي البرمجة؟ البرمجة هي فن وعلم إعطاء الأوامر للحاسوب ليقوم بتنفيذ مهام محددة. تخيل أنك تتحدث إلى آلة، وتخبرها خطوة بخطوة ماذا تفعل. هذه الأوامر تُكتب بلغة يفهمها الحاسوب، وتسمى "لغة برمجة". الحواسيب لا تفهم لغتنا البشرية مباشرة، لذا نستخدم لغات البرمجة كوسيط. المبرمج يكتب الكود، ثم تقوم أداة خاصة (مثل المترجم أو المفسر) بتحويل هذا الكود إلى تعليمات يفهمها الحاسوب. ### 1.2 لماذا "بَرمَجة"؟ تعلم البرمجة قد يكون صعبًا في البداية، خاصة وأن معظم لغات البرمجة تستخدم الكلمات الإنجليزية. "بَرمَجة" جاءت لتحل هذه المشكلة: * **سهولة التعلم**: تستخدم كلمات عربية مألوفة (مثل `اطبع`، `إذا`، `لكل`) بدلاً من الكلمات الإنجليزية (مثل `for`، `if`، `print`). * **كسر حاجز اللغة**: لا حاجة لتعلم مصطلحات إنجليزية معقدة في البداية، مما يسرع عملية الفهم والتعلم. * **قوة JavaScript**: "بَرمَجة" تتحول إلى JavaScript، مما يعني أنك تستفيد من قوة JavaScript الهائلة وقدرتها على بناء أي شيء تقريبًا (مواقع ويب، تطبيقات جوال، خوادم). * **بيئة تطوير متكاملة**: تأتي مع إضافة لـ VS Code توفر لك تجربة برمجة احترافية (تلوين الكود، إكمال تلقائي، تشغيل مباشر). ## الفصل الثاني: البدء مع "بَرمَجة" وبيئة التطوير ### 2.1 تثبيت VS Code Visual Studio Code (VS Code) هو محرر أكواد مجاني وقوي، وهو البيئة التي سنستخدمها لكتابة كود "بَرمَجة". 1. قم بزيارة الموقع الرسمي: code.visualstudio.com 2. قم بتنزيل وتثبيت النسخة المناسبة لنظام التشغيل الخاص بك (Windows, macOS, Linux). ### 2.2 تثبيت إضافة "بَرمَجة" لـ VS Code بعد تثبيت VS Code، ستحتاج إلى تثبيت إضافة "بَرمَجة" التي توفر الدعم الكامل للغة: 1. افتح VS Code. 2. انقر على أيقونة "Extensions" (المربعات الأربعة) في الشريط الجانبي الأيسر، أو اضغط `Ctrl+Shift+X`. 3. في شريط البحث، اكتب `Barmaja`. 4. ابحث عن الإضافة التي تحمل اسم `Barmaja Language Support` (أو ما شابه) وانقر على زر `Install`. ### 2.3 كتابة وتشغيل أول برنامج لك: "مرحباً بالعالم!" الآن أنت جاهز لكتابة أول برنامج لك بلغة "بَرمَجة"! 1. في VS Code، أنشئ ملفًا جديدًا (File > New File). 2. احفظ الملف باسم `hello.brm` (امتداد `.brm` هو لملفات "بَرمَجة"). 3. اكتب الكود التالي في الملف: ```barmaja // هذا أول برنامج لنا بلغة بَرمَجة اطبع("مرحباً بالعالم من بَرمَجة!"); ```. 4. لتشغيل الكود: * افتح لوحة الأوامر (Command Palette) بالضغط على `Ctrl+Shift+P`. * اكتب `Barmaja: Run Code` واختر الأمر. * سترى النتيجة في نافذة Terminal أسفل VS Code. ![مرحباً بالعالم في VS Code](home/ubuntu/barmaja_hello_world_vscode.png/) *الشكل 2.1: أول

برنامج "مرحباً بالعالم" بلغة بَرمَجة في VS Code.* ## الفصل الثالث: أساسيات اللغة: المتغيرات وأنواع البيانات ### 3.1 المتغيرات (Variables) المتغير هو مكان في الذاكرة لتخزين البيانات. تخيل أنه صندوق يمكنك وضع أي شيء بداخله وتغيير محتوياته لاحقًا. في "بَرمَجة"، نستخدم الكلمة المفتاحية `دع` لتعريف المتغيرات. **مثال:** ```barmaja دع اسم_الطالب = "أحمد"; // قمنا بتعريف متغير اسمه "اسم_الطالب" وخزنا فيه قيمة نصية "أحمد" اطبع("اسم الطالب هو: " + اسم_الطالب); اسم_الطالب = "سارة"; // يمكننا تغيير قيمة المتغير لاحقًا اطبع("الاسم الجديد: " + اسم_الطالب); ``` ### 3.2 الثوابت (Constants) الثابت هو أيضًا مكان لتخزين البيانات، لكن قيمته لا يمكن تغييرها بعد تعريفها. تخيل أنه صندوق تضع فيه شيئًا ثم تغلقه بإحكام ولا يمكنك فتحه مرة أخرى. نستخدم الكلمة المفتاحية `ثابت` لتعريف الثوابت. **مثال:** ```barmaja ثابت عدد_الأيام_في_الأسبوع = 7; اطبع("عدد الأيام في الأسبوع: " + عدد_الأيام_في_الأسبوع); // عدد_الأيام_في_الأسبوع = 8; // إذا حاولت تغيير قيمة الثابت، سيظهر لك خطأ! ```
![المتغيرات والثوابت في VS Code](/home/ubuntu/barmaja_variables_vscode.png) *الشكل 3.1: تعريف المتغيرات والثوابت في لغة بَرمَجة.* ### 3.3 أنواع البيانات (Data Types) البيانات التي نخزنها في المتغيرات أو الثوابت لها أنواع مختلفة. فهم هذه الأنواع مهم جدًا لأنها تحدد العمليات التي يمكن إجراؤها عليها. #### 3.3.1 الأرقام (Numbers) تستخدم لتخزين الأعداد الصحيحة والعشرية. **مثال:** ```barmaja دع العمر = 15; دع السعر = 99.99; ``` #### 3.3.2 النصوص (Strings) تستخدم لتخزين النصوص أو مجموعة من الحروف. توضع النصوص بين علامتي اقتباس مفردة (`'`) أو مزدوجة (`"`). **مثال:** ```barmaja دع رسالة_ترحيب = "أهلاً بك في بَرمَجة!"; دع حرف_واحد = 'أ'; ``` #### 3.3.3 القيم المنطقية (Booleans) تستخدم لتخزين قيمتين فقط: `صحيح` (True) أو `خطأ` (False). وهي أساس اتخاذ القرارات في البرمجة. **مثال:** ```barmaja دع هل_الجو_جميل = صحيح; دع هل_الامتحان_صعب = خطأ; ``` #### 3.3.4 القيم الخاصة: `فارغ` (Null) و `غير_معرف` (Undefined) * `فارغ`: تعني أن المتغير لا يحتوي على أي قيمة بشكل متعمد. * `غير_معرف`: تعني أن المتغير تم تعريفه ولكن لم يتم إسناد أي قيمة له. **مثال:** ```barmaja دع لا_شيء = فارغ; دع قيمة_غير_محددة; // لم يتم إسناد قيمة لها، لذا ستكون غير_معرف ``` #### 3.3.5 المصفوفات (Arrays) تستخدم لتخزين قائمة من القيم (يمكن أن تكون من نفس النوع أو أنواع مختلفة) في متغير واحد. يتم الوصول إلى عناصر المصفوفة باستخدام الفهرس (Index) الذي يبدأ من الصفر. **مثال:** ```barmaja دع قائمة_فواكه = ["تفاح", "برتقال", "موز"]; اطبع("الفاكهة الأولى: " + قائمة_فواكه[0]); // الناتج: تفاح ``` #### 3.3.6 الكائنات (Objects) تستخدم لتخزين مجموعة من الخصائص (Properties) والقيم المرتبطة بها. كل خاصية لها اسم (مفتاح) وقيمة. **مثال:** ```barmaja دع طالب = { اسم: "ليلى", عمر: 16, تخصص: "علمي" }; اطبع("اسم الطالبة: " + طالب.اسم); // الناتج: ليلى ```
![أنواع البيانات في VS Code](/home/ubuntu/barmaja_data_types_vscode.png) *الشكل 3.2: أنواع البيانات المختلفة في لغة بَرمَجة.* ## الفصل الرابع: المعاملات (Operators) المعاملات هي رموز تستخدم لإجراء عمليات على القيم والمتغيرات. ### 4.1 المعاملات الحسابية (Arithmetic Operators) تستخدم لإجراء العمليات الحسابية الأساسية. | المعامل | الوصف | مثال | الناتج | | --- | --- | --- | --- | | `+` | الجمع | `5 + 3` | `8` | | `-` | الطرح | `5 - 3` | `2` | | `*` | الضرب | `5 * 3` | `15` | | `/` | القسمة | `10 / 3` | `3.33` | | `%` | باقي القسمة | `10 % 3` | `1` | | `**` | الأس | `2 ** 3` (3^2) |

`8` | **مثال:** ```barmaja دع س = 10; دع ص = 3; اطبع("الجمع: " + (س + ص)); // 13
اطبع("القسمة: " + (س / ص)); // 3.333... ``` ### 4.2 معاملات الإسناد (Assignment Operators)
تستخدم لإسناد قيمة لمتغير. | المعامل | مثال | يعادل | | :----- | :---------- | :---------- | | `=` |
`دع س = 5;` | | | `+=` | `س += 3;` | `س = س + 3;` | | `-=` | `س -= 3;` | `س = س - 3;` | | `*=` |
`س *= 3;` | `س = س * 3;` | | `/=` | `س /= 3;` | `س = س / 3;` | | `%=` | `س %= 3;` | `س = س
% 3;` | | `**=` | `س **= 3;` | `س = س ** 3;` | **مثال:** ```barmaja دع عدد = 5; عدد += 3; //
الآن عدد = 8 اطبع("عدد بعد +=: " + عدد); ``` ### 4.3 معاملات المقارنة (Comparison
Operators) تستخدم لمقارنة قيمتين، والنتيجة دائمًا تكون `صحيح` أو `خطأ`. | المعامل | الوصف |
مثال | الناتج | | :----- | :---------- | :---------- | :----- | | `==` | يساوي (قيمة فقط) | `5
== "5"` | `صحيح` | | `===` | يساوي (قيمة ونوع) | `5 === "5"` | `خطأ` | | `!=` | لا يساوي (قيمة
فقط) | `5 != "5"` | `خطأ` | | `!==` | لا يساوي (قيمة ونوع) | `5 !== "5"` | `صحيح` | | `<` | أكبر من |
`10 < 5` | `صحيح` | | `>` | أصغر من | `10 > 5` | `خطأ` | | `>=` | أكبر من أو يساوي | `10 >= 10` |
`صحيح` | | `<=` | أصغر من أو يساوي | `10 <= 5` | `خطأ` | **مثال:** ```barmaja دع عمر_أحمد =
20; اطبع("هل عمر أحمد أكبر من 18؟ " + (عمر_أحمد > 18)); // صحيح ``` ### 4.4 المعاملات
المنطقية (Logical Operators) تستخدم للجمع بين الشروط المنطقية. | المعامل | الوصف | مثال
| الناتج | | :----- | :---------- | :----- | | `&&` | و (AND) | `صحيح && خطأ` | `خطأ` | |
`||` | أو (OR) | `صحيح || خطأ` | `صحيح` | | `!` | ليس (NOT) | `!صحيح` | `خطأ` | **مثال:**
```barmaja دع هل_طالب = صحيح; دع هل_ناجح = خطأ; اطبع("هل طالب وناجح؟ " + (هل_طالب
&& هل_ناجح)); // خطأ ``` ![المعاملات في VS Code
(/home/ubuntu/barmaja_operators_vscode.png) *الشكل 4.1: استخدام المعاملات المختلفة
في لغة بَرمَجة.* ## الفصل الخامس: الجمل الشرطية (Conditional Statements) الجمل الشرطية
تسمح لبرنامجك باتخاذ قرارات بناءً على شروط معينة. إذا كان الشرط صحيحًا، يتم تنفيذ جزء من
الكود، وإلا يتم تنفيذ جزء آخر. ### 5.1 جملة `إذا` (If Statement) تستخدم لتنفيذ كود معين إذا
كان الشرط صحيحًا. **الصيغة العامة:** ```barmaja إذا (الشرط_هنا) { // الكود الذي سيتم تنفيذه
إذا كان الشرط صحيحًا } ``` **مثال:** ```barmaja دع درجة = 85; إذا (درجة >= 50) { اطبع("لقد
نجحت!"); } ``` ### 5.2 جملة `إذا` و `وإلا` (If-Else Statement) تستخدم لتنفيذ كود إذا كان الشرط
صحيحًا، وكود آخر إذا كان الشرط خاطئًا. **الصيغة العامة:** ```barmaja إذا (الشرط_هنا) { // الكود
إذا كان الشرط صحيحًا } وإلا { // الكود إذا كان الشرط خاطئًا } ``` **مثال:** ```barmaja دع درجة =
45; إذا (درجة >= 50) { اطبع("لقد نجحت!"); } وإلا { اطبع("لم تنجح."); } ``` ### 5.3 جملة `إذا` و
`وإلا إذا` و `وإلا` (If-Else If-Else Statement) تستخدم لاختبار عدة شروط بالتسلسل. **الصيغة
العامة:** ```barmaja إذا (الشرط_الأول) { // الكود إذا كان الشرط الأول صحيحًا } وإلا إذا
(الشرط_الثاني) { // الكود إذا كان الشرط الثاني صحيحًا } وإلا { // الكود إذا لم يكن أي من الشروط
السابقة صحيحًا } ``` **مثال:** ```barmaja دع درجة_الطالب = 75; إذا (درجة_الطالب >= 90) {
اطبع("التقدير: ممتاز"); } وإلا إذا (درجة_الطالب >= 80) { اطبع("التقدير: جيد جداً"); } وإلا إذا
(درجة_الطالب >= 70) { اطبع("التقدير: جيد"); } وإلا { اطبع("التقدير: ضعيف"); } ``` ![الجمل
الشرطية في VS Code](/home/ubuntu/barmaja_conditions_vscode.png) *الشكل 5.1:
استخدام الجمل الشرطية في لغة بَرمَجة.* ## الفصل السادس: الحلقات التكرارية (Loops)
```

الحلقات التكرارية تسمح لك بتنفيذ جزء من الكود عدة مرات. ### 6.1 حلقة `لكل` (For Loop) تستخدم عندما تعرف عدد مرات التكرار مسبقًا. **الصيغة العامة:** ```barmaja لكل (التهيئة; الشرط; التحديث) { // الكود الذي سيتم تكراره } ``` **مثال:** طباعة الأرقام من 0 إلى 4. ```barmaja لكل (دع i = 0; i < 5; ++i) { اطبع("الرقم الحالي: " + i); } ``` ### 6.2 حلقة `طالما` (While Loop) تستخدم عندما لا تعرف عدد مرات التكرار مسبقًا، وتستمر الحلقة طالما كان الشرط صحيحًا. **الصيغة العامة:** ```barmaja طالما (الشرط_هنا) { // الكود الذي سيتم تكراره طالما الشرط صحيح } ``` **مثال:** طباعة العداد حتى يصل إلى 3. ```barmaja دع عداد = 0; طالما (عداد < 3) { اطبع("العداد: " + عداد); عداد++; // مهم جدًا تحديث الشرط لتجنب حلقة لا نهائية! } ``` ![الحلقات التكرارية في VS Code](/home/ubuntu/barmaja_loops_vscode.png) *الشكل 6.1: استخدام الحلقات التكرارية في لغة بَرمَجة.* ## الفصل السابع: الدوال (Functions) الدالة هي كتلة من الكود تؤدي مهمة محددة. يمكنك تعريف الدالة مرة واحدة واستدعائها عدة مرات، مما يجعل الكود أكثر تنظيمًا وقابلية لإعادة الاستخدام. ### 7.1 تعريف واستدعاء الدوال نستخدم الكلمة المفتاحية `دالة` لتعريف الدالة. **الصيغة العامة:** ```barmaja دالة اسم_الدالة(معامل1, معامل2) { // الكود الذي تنفذه الدالة أرجع قيمة; // اختياري: لإرجاع قيمة من الدالة } // استدعاء الدالة اسم_الدالة(قيمة1, قيمة2); ``` **مثال:** دالة بسيطة لطباعة رسالة. ```barmaja دالة أهلاً() { اطبع("أهلاً بك في عالم الدوال!"); } أهلاً(); // استدعاء الدالة ``` **مثال:** دالة تستقبل معاملات. ```barmaja دالة ترحيب_باسم(اسم) { اطبع("مرحباً يا " + اسم + "!"); } ترحيب_باسم("فاطمة"); ``` **مثال:** دالة ترجع قيمة. ```barmaja دالة جمع_رقمين(رقم1, رقم2) { دع المجموع = رقم1 + رقم2; أرجع المجموع; } دع نتيجة_الجمع = جمع_رقمين(10, 5); اطبع("نتيجة الجمع: " + نتيجة_الجمع); ``` ![الدوال في VS Code](/home/ubuntu/barmaja_functions_vscode.png) *الشكل 7.1: تعريف واستدعاء الدوال في لغة بَرمَجة.* ## الفصل الثامن: البرمجة كائنية التوجه (Object-Oriented Programming - OOP) البرمجة كائنية التوجه هي نموذج برمجي يعتمد على مفهوم "الكائنات"، والتي يمكن أن تحتوي على بيانات (خصائص) ووظائف (دوال). هذا النموذج يساعد على تنظيم الكود وجعله أكثر قابلية للإدارة وإعادة الاستخدام. ### 8.1 الفئات (Classes) والكائنات (Objects) * **الفئة (Class)**: هي قالب أو مخطط لإنشاء الكائنات. تحدد الخصائص والدوال التي ستمتلكها الكائنات التي تنشأ منها. * **الكائن (Object)**: هو نسخة حقيقية من الفئة. يمكنك إنشاء العديد من الكائنات من نفس الفئة، وكل كائن له خصائصه الخاصة. نستخدم الكلمة المفتاحية `صنف` لتعريف الفئات، و `جديد` لإنشاء كائنات جديدة. **الصيغة العامة:** ```barmaja صنف اسم_الفئة { جديد(معاملات) { // الدالة البانية (Constructor) هذا.خاصية = قيمة; } دالة اسم_الدالة() { // كود الدالة } } دع كائن = جديد اسم_الفئة(قيم_المعاملات); ``` **مثال:** فئة `شخص`. ```barmaja صنف شخص { جديد(اسم, عمر) { هذا.اسم = اسم; هذا.عمر = عمر; } دالة قدم_نفسك() { أرجع "أهلاً، أنا " + هذا.اسم + " وعمري " + هذا.عمر + " سنة."; } } دع طالب1 = جديد شخص("علي", 22); اطبع(طالب1.قدم_نفسك()); // أهلاً، أنا علي وعمري 22 سنة. ``` ### 8.2 الوراثة (Inheritance) الوراثة تسمح لفئة (الفئة الابنة) بأن ترث الخصائص والدوال من فئة أخرى (الفئة الأم). هذا يعزز إعادة استخدام الكود ويقلل التكرار. نستخدم الكلمة المفتاحية `يمتد` للوراثة، و `فائق` لاستدعاء دالة الفئة الأم. **مثال:** فئة `معلم` ترث من `شخص`. ```barmaja صنف معلم يمتد شخص {

جديد(اسم, عمر, تخصص) { فائق(اسم, عمر); // استدعاء الدالة البانية للفئة الأم هذا.تخصص = تخصص; } دالة قدم_نفسك() { أرجع فائق.قدم_نفسك() + " وأنا أدرّس " + هذا.تخصص + "."; } } دع معلم1 = جديد معلم("أستاذ خالد", 40, "الرياضيات"); اطبع(معلم1.قدم_نفسك()); // أهلاً، أنا أستاذ خالد وعمري 40 سنة. وأنا أدرّس الرياضيات. ``` ![الفئات والكائنات في VS Code] (home/ubuntu/barmaja_classes_vscode.png/) *الشكل 8.1: تعريف الفئات والكائنات والوراثة في لغة بَرمَجة.* ## الفصل التاسع: تطوير الويب باستخدام "بَرمَجة" الآن بعد أن أصبحت ملمًا بأساسيات "بَرمَجة"، حان الوقت لتطبيقها في بناء صفحات الويب التفاعلية. بما أن "بَرمَجة" تتحول إلى JavaScript، يمكنها التفاعل مع عناصر صفحة الويب (DOM) تمامًا مثل JavaScript. ### 9.1 محرك تشغيل "بَرمَجة" في المتصفح (Barmaja Runtime) لجعل كود "بَرمَجة" يعمل مباشرة في المتصفح، قمنا بتطوير محرك تشغيل صغير. كل ما عليك فعله هو تضمين هذا المحرك في صفحة HTML الخاصة بك، ثم كتابة كود "بَرمَجة" داخل وسم ` ``` ![مثال ويب بلغة بَرمَجة في المتصفح] (home/ubuntu/barmaja_web_example_browser.png/) *الشكل 9.1: صفحة ويب بسيطة تعمل بلغة بَرمَجة مباشرة في المتصفح.* ### 9.2 التفاعل مع DOM (Document Object Model) يمكن لـ "بَرمَجة" التفاعل مع عناصر صفحة الويب (HTML) تمامًا مثل JavaScript. يمكنك تغيير النصوص، الألوان، إضافة عناصر جديدة، والاستجابة لأحداث المستخدم (مثل النقر على زر). **مثال:** عداد نقرات بسيط. ```html

تفاعل DOM باستخدام بَرمَجة

انقر هنا!

عدد النقرات: 0

```
```
![تفاعل DOM بلغة بَرمَجة في المتصفح]
(home/ubuntu/barmaja_dom_interaction_browser.png/) *الشكل 9.2: صفحة ويب تفاعلية تستخدم "بَرمَجة" للتفاعل مع DOM.* ### 9.3 إنشاء مشروع ويب جديد باستخدام إضافة VS Code يمكنك البدء بمشروع ويب جديد بسرعة باستخدام أمر خاص في إضافة "بَرمَجة": 1. افتح لوحة الأوامر (Command Palette) بالضغط على `Ctrl+Shift+P`. 2. اكتب `Barmaja: Create Web Project` واختر الأمر. 3. سيتم إنشاء مجلد جديد يحتوي على ملف `index.html` وملف `barmaja-runtime.js` جاهزين للعمل. ![إنشاء مشروع ويب بلغة بَرمَجة في VS Code] (home/ubuntu/barmaja_create_web_project_vscode.png/) *الشكل 9.3: استخدام أمر "Barmaja: Create Web Project" في VS Code.* ## الفصل العاشر: نشر إضافة "بَرمَجة" على VS Code Marketplace لنشر إضافة "بَرمَجة" الخاصة بك على متجر VS Code (Marketplace)، ستحتاج إلى اتباع الخطوات التالية. هذه الخطوات تسمح للمستخدمين الآخرين بتثبيت الإضافة مباشرة من VS Code. ### 10.1 المتطلبات الأساسية قبل البدء، تأكد من تثبيت Node.js و npm على جهازك.

ثم قم بتثبيت أداة `(Visual Studio Code Extension) vsce` عالميًا: ```bash npm install -g vsce
```
![تثبيت vsce](/home/ubuntu/barmaja_vsce_install.png) *الشكل 10.1: تثبيت أداة `vsce` عبر سطر الأوامر.* ### 10.2 إنشاء ناشر (Publisher) في Visual Studio Marketplace للنشر، يجب أن يكون لديك حساب ناشر. إذا لم يكن لديك، اتبع الخطوات التالية: 1. انتقل إلى [Visual Studio Marketplace Publisher Management](https://marketplace.visualstudio.com/manage/publishers). 2. سجل الدخول باستخدام حساب Microsoft الخاص بك. 3. انقر على `Create new publisher`. 4. أدخل `Publisher ID` (معرف فريد، مثل `alaa-saber`) و `Publisher Name` (اسم العرض، مثل `Alaa Saber`). 5. انقر على `Create Publisher`. ![إنشاء ناشر في Marketplace](/home/ubuntu/barmaja_marketplace_publisher.png) *الشكل 10.2: إنشاء ناشر جديد في Visual Studio Marketplace.* ### 10.3 إنشاء رمز وصول شخصي (Personal Access Token - PAT) ستحتاج إلى PAT للسماح لأداة `vsce` بالوصول إلى حساب الناشر الخاص بك ونشر الإضافة. 1. انتقل إلى [Azure DevOps](https://dev.azure.com/) وسجل الدخول. 2. في الزاوية العلوية اليمنى، انقر على أيقونة المستخدم الخاص بك، ثم اختر `Personal access tokens`. 3. انقر على `New Token`. 4. أدخل اسمًا للرمز (مثلاً `VSCodePublisherToken`). 5. اختر المنظمة التي أنشأت فيها الناشر (إذا كانت موجودة). 6. في قسم `Scopes`، اختر `Custom defined`، ثم انقر على `Show all scopes`. 7. ابحث عن `Marketplace` وقم بتحديد `Publish`. 8. انقر على `Create`. 9. **انسخ الرمز الذي يظهر لك فورًا!** لن تتمكن من رؤيته مرة أخرى. ![إنشاء PAT في Azure DevOps](/home/ubuntu/barmaja_azure_devops_pat.png) *الشكل 10.3: إنشاء Personal Access Token (PAT) مع صلاحية النشر للمتجر.* ### 10.4 تسجيل الدخول إلى `vsce` باستخدام PAT الآن، استخدم `vsce` لتسجيل الدخول إلى حساب الناشر الخاص بك باستخدام الـ PAT الذي أنشأته: 1. افتح Terminal في مجلد إضافة VS Code (المجلد الذي يحتوي على `package.json`). 2. نفذ الأمر التالي، مع استبدال `your-publisher-id` بمعرف الناشر الخاص بك: ```bash vsce login your-publisher-id ```. 3. عندما يطلب منك `Personal Access Token:`, الصق الرمز الذي نسخته في الخطوة السابقة واضغط Enter. ```
bash > vsce login alaa-saber
Publisher 'alaa-saber'
Personal Access Token: [**********] was successfully added
```
![تسجيل الدخول إلى vsce](/home/ubuntu/barmaja_vsce_login.png) *الشكل 10.4: تسجيل الدخول إلى `vsce` باستخدام Personal Access Token.* ### 10.5 حزم الإضافة (Packaging the Extension) قبل النشر، يمكنك حزم الإضافة في ملف `.vsix` لاختبارها محليًا أو مشاركتها يدويًا. 1. في Terminal، تأكد أنك في مجلد إضافة VS Code. 2. نفذ الأمر: ```
bash vsce package
```. 3. سيتم إنشاء ملف `.vsix` في نفس المجلد. ```
bash ### Packaging ...
... (رسائل أخرى)
Successfully packaged extension barmaja-vscode-extension-1.0.0.vsix!
```
![حزم الإضافة](/home/ubuntu/barmaja_vsce_package.png) *الشكل 10.5: حزم الإضافة في ملف `.vsix`.* ### 10.6 نشر الإضافة (Publishing the Extension) أخيرًا، لنشر الإضافة على Visual Studio Marketplace: 1. في Terminal، تأكد أنك في مجلد إضافة VS Code. 2. نفذ الأمر: ```
bash vsce publish
```. 3. سيتم رفع الإضافة إلى Marketplace، وستصبح متاحة للجميع بعد بضع دقائق.
```

```bash Publishing extension... Successfully published barmaja-vscode-extension to Visual Studio Marketplace. ```
![نشر الإضافة](/home/ubuntu/barmaja_vsce_publish.png)
الشكل 10.6: نشر الإضافة على Visual Studio Marketplace. تهانينا! لقد قمت بنشر إضافة "بَرمَجة" الخاصة بك. يمكن للمستخدمين الآن البحث عنها وتثبيتها مباشرة من VS Code. ## الخاتمة لقد أكملت رحلتك في تعلم أساسيات البرمجة بلغة "بَرمَجة"، وأصبحت الآن قادرًا على بناء تطبيقات بسيطة، والتفاعل مع الويب، وحتى نشر أدواتك الخاصة. تذكر أن البرمجة هي مهارة تتطور بالممارسة المستمرة. استمر في التجربة، بناء المشاريع، ولا تخف من ارتكاب الأخطاء، فهي جزء أساسي من عملية التعلم. نتمنى لك كل التوفيق في مسيرتك البرمجية!

الفصل الحادي عشر: المكتبات المعربة المتقدمة

لجعل لغة "بَرمَجة" لغة متكاملة، قمنا بتعريب أهم المكتبات البرمجية التي يحتاجها المبرمج يومياً. إليك قائمة بأهم هذه المكتبات وكيفية استخدامها:

11.1 مكتبة الرياضيات (الرياضيات)

بدلاً من استخدام `Math` في JavaScript، يمكنك الآن استخدام `الرياضيات` بكلمات عربية واضحة:

- `الرياضيات.عشوائي()` : توليد رقم عشوائي بين 0 و 1.
- `الرياضيات.جذر(16)` : حساب الجذر التربيعي (الناتج: 4).
- `الرياضيات.قوة(2, 3)` : حساب الأس (2^3 = 8).
- `الرياضيات.تقريب(4.7)` : تقريب الرقم لأقرب عدد صحيح (الناتج: 5).
- `الرياضيات.أرضية(4.7)` : التقريب للأسفل (الناتج: 4).
- `الرياضيات.سقف(4.2)` : التقريب للأعلى (الناتج: 5).

مثال:

```
دع رقم_عشوائي = الرياضيات.عشوائي() * 10;
اطبع("رقم عشوائي مقرب: " + الرياضيات.تقريب(رقم_عشوائي));
```

11.2 مكتبة التاريخ (التاريخ)

للتعامل مع الوقت والتاريخ بسهولة:

- `التاريخ.الآن()` : الحصول على الوقت الحالي بالملي ثانية.
- `دع اليوم = جديد التاريخ()` : إنشاء كائن تاريخ جديد.
```

- `اليوم.احصل_على_السنة()` : الحصول على السنة الحالية.
- `اليوم.احصل_على_الشهر()` : الحصول على الشهر (0-11).

11.3 مكتبة النصوص

أصبح التعامل مع النصوص أكثر سهولة:

- `نص.طول` : الحصول على عدد حروف النص.
- `نص.إلى_كبير()` : تحويل الحروف اللاتينية لكبيرة.
- `نص.إلى_صغير()` : تحويل الحروف اللاتينية لصغيرة.
- `نص.تضمن("كلمة")` : التأكد من وجود كلمة داخل النص.

الفصل الثاني عشر: صفحة الهبوط والموارد الرسمية

للحصول على أفضل تجربة مع لغة "بَرمَجة"، قمنا بإنشاء صفحة هبوط (Landing Page) رسمية تجمع كل ما تحتاجه في مكان واحد.

12.1 محتويات صفحة الهبوط

تضم الصفحة:

1. **الملعب التفاعلي**: لتجربة الكود مباشرة دون تثبيت.
2. **روابط التحميل**: لتحميل هذا الكتاب وملف الإضافة `vsix.` .
3. **شرح المميزات**: عرض سريع لقوة اللغة وإمكانياتها.
4. **مقارنة الأكواد**: لترى الفرق بين "بَرمَجة" و JavaScript التقليدية.

12.2 كيفية استخدام ملف `vsix.`

إذا قمت بتحميل ملف الإضافة `barmaja-extension.vsix` من الموقع، يمكنك تثبيته في VS Code يدوياً:

1. افتح VS Code.
2. انتقل إلى قسم `Extensions` .
3. انقر على النقاط الثلاث `...` في الأعلى.

4. اختر `Install from VSIX...`.
5. حدد الملف الذي قمت بتحميله، وسيتم تثبيت الإضافة فوراً!

تم تحديث هذا المنهج ليشمل كافة الميزات المتقدمة لعام 2026. حقوق الصانع: علاء صابر